

# Aprendizado por Reforço

Edimilson B. dos Santos

# Introdução

- Aprendizado por Reforço:
  - Trata a questão de como um agente autônomo que sente e age em seu ambiente pode aprender a escolher ações ótimas para alcançar seus objetivos.
  - Cobre tarefas como aprender a controlar um robô móvel, a otimizar operações em fábricas e a jogar jogos de tabuleiro.
  - Cada vez que o agente executa uma ação, o treinador pode fornecer uma recompensa ou penalidade para indicar a conveniência do estado resultante.
    - Ex: Ao treinar um agente para jogar um jogo, esse agente pode ter recompensa positiva se ganhar, negativa se perder ou zero em outros casos.

# Introdução

- Considere construir um robô de aprendizagem.
- O robô, ou agente, tem:
  - Um conjunto de sensores para observar os estados de seu ambiente;
  - Um conjunto de ações que pode executar para alterar seu estado.
  - Ex: um robô móvel pode ter sensores, como uma câmera e sonares; e ações como “mover-se para frente” e “girar”.
- A tarefa do robô, ou agente, é aprender uma estratégia de controle, ou política, para escolher ações que alcancem seus objetivos.
  - Ex: o robô pode ter um objetivo de colocar seu carregador de bateria sempre que seu nível de bateria está baixo.

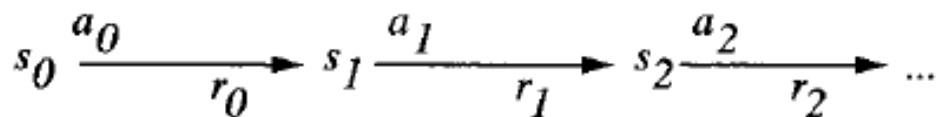
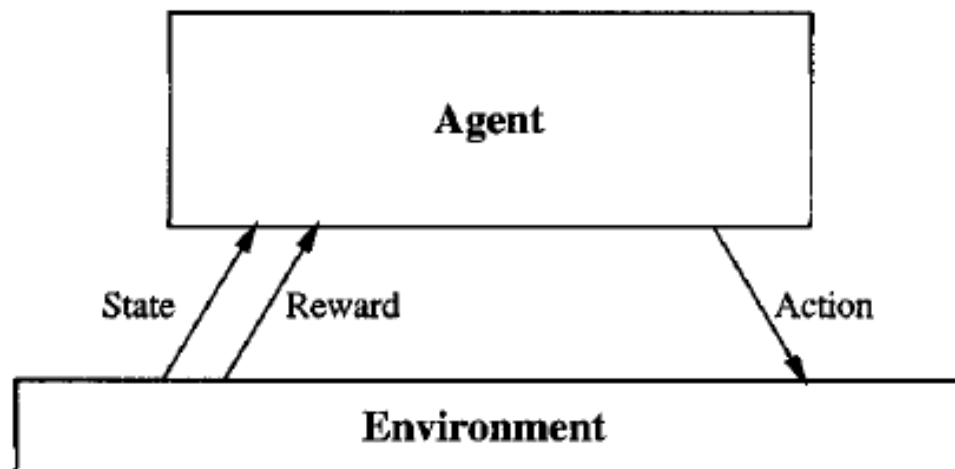
# Introdução

- **Interesse:** como os agentes podem aprender, com sucesso, políticas de controle explorando seu ambiente?
- Assume-se que os objetivos do agente podem ser definidos por uma **função de recompensa**: atribui um valor numérico para cada ação distinta que o agente pode executar a partir de cada estado distinto.
  - Ex: o objetivo de colocar o carregador de bateria pode ser capturado atribuindo:
    - uma recompensa positiva (ex, +100) para transições de estado-ação que resultam em uma conexão para o carregador;
    - Uma recompensa zero para outras transições estado-ação.
- A função de recompensa pode ser construída dentro do robô ou conhecida apenas por um orientador externo.

# Introdução

- A tarefa do robô é executar:
  - sequências de ações;
  - observar suas consequências; e
  - aprender uma política de controle.
- A política de controle desejada é aquela que, a partir de qualquer estado inicial, escolhe ações que maximizam a recompensa acumulada durante o tempo pelo agente.

# Introdução



Goal: Learn to choose actions that maximize

$$r_0 + \gamma r_1 + \gamma^2 r_2 + \dots, \text{ where } 0 \leq \gamma < 1$$

Cada vez que o agente executa uma ação  $a_t$  em algum estado  $s_t$ , o agente recebe uma recompensa  $r_t$  (valor real). Isso produz uma sequência de estados  $s_i$ , ações  $a_i$  e recompensas  $r_i$ . A tarefa do agente é aprender uma política de controle,  $\pi: S \rightarrow A$ , que maximiza a soma esperada dessas recompensas, com recompensas futuras descontadas exponencialmente por seus atrasos.

# Introdução

- O problema de aprender uma política de controle cobre muitos problemas.
  - É um problema de aprender a controlar processos sequenciais.
  - Ex: problemas de otimização de produção industrial, problemas de escalonamento sequencial.
- Dentro desta classe de problemas, nós consideramos configurações específicas, incluindo:
  - Configurações nas quais as ações tem resultados determinísticos e não-determinísticos.
  - Configurações nos quais os agentes tem ou não conhecimento a priori sobre os efeitos de suas ações no ambiente.

# Introdução

- O problema de aprender uma política de controle para escolher ações é parecido com o problema de aproximação de funções.
  - A função objetivo a ser aprendida neste caso é uma política de controle,  $\pi: S \rightarrow A$ .



# Introdução

- Contudo, o problema de aprendizado por reforço difere de outras tarefas de aproximação de funções em vários aspectos importantes:
  - Recompensa atrasada: a informação de treinamento não está disponível e o agente enfrenta o problema de determinar quais ações produzem recompensas.
  - Exploração: descobrir qual estratégia produz aprendizado mais eficiente.
  - Estados observáveis parcialmente: em muitas situações práticas os sensores fornecem apenas informação parcial.
  - Aprendizado de longa vida: aprendizado de robô requer que o robô aprenda várias tarefas relacionadas dentro do mesmo ambiente, usando os mesmos sensores.

# A Tarefa do Aprendizado

- Há muitas maneiras de formular o problema de aprender estratégias de controle sequenciais:
  - Assumir que as ações do agente são determinísticas ou não;
  - Assumir que o agente pode prever o próximo estado que resultará de cada ação, ou não, que ele não pode.
  - Assumir que o agente é treinado por um especialista que mostra a ele exemplos de sequências de ações ótimas, ou que ele deve treinar a si próprio executando ações de sua escolha própria.
- Aqui, é definida uma formulação mais geral, com base nos Processos de Decisão de Markov (MDP).

# A Tarefa do Aprendizado

## Processos de Decisão de Markov (MDP)

- Em um MDP, o agente pode observar um conjunto  $S$  de estados distintos de seu ambiente e tem um conjunto  $A$  de ações que ele pode executar.
- Em cada passo de tempo  $t$ , o agente sente o estado atual  $s_t$ , escolhe uma ação atual  $a_t$  e a executa.
- O ambiente responde dando ao agente uma recompensa  $r_t = r(s_t, a_t)$  e produzindo o estado sucessor  $s_{t+1} = \delta(s_t, a_t)$ .
- As funções  $\delta$  e  $r$  são partes do ambiente e não são necessariamente conhecidas para o agente.
  - Em um MDP, as funções  $r(s_t, a_t)$  e  $\delta(s_t, a_t)$  dependem apenas do estado e ação atuais e não de estados e ações anteriores.

# A Tarefa do Aprendizado

- A tarefa do agente é aprender uma política,  $\pi: S \rightarrow A$ , para selecionar sua próxima ação  $a_t$  com base no estado atual observado  $s_t$ ; ou seja,  $\pi(s_t) = a_t$ .
- Como especificar qual política  $\pi$  gostaríamos que o agente aprendesse?
- Uma abordagem é requisitar a política que produz a maior recompensa acumulada possível para o robô durante o tempo.

# A Tarefa do Aprendizado

Nós definimos o valor acumulado  $V^\pi(s_t)$ :

$$\begin{aligned} V^\pi(s_t) &\equiv r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \dots \\ &\equiv \sum_{i=0}^{\infty} \gamma^i r_{t+i} \end{aligned}$$

onde a sequência de recompensas  $r_{t+i}$  é gerada iniciando no estado  $s_t$  e usando a política para selecionar ações.

Aqui  $0 \leq \gamma \leq 1$  é uma constante que determina o valor relativo de recompensas imediatas versus atrasadas.

# A Tarefa do Aprendizado

- A quantidade  $V^\pi(s_t)$  é frequentemente chamada de *recompensa acumulada com desconto* alcançada pela política  $\pi$  a partir do estado inicial  $s$ .
- É razoável descontar recompensas futuras em relação à recompensas imediatas.
  - Em muitos casos, nós preferimos obter a recompensa mais cedo do que mais tarde.

# A Tarefa do Aprendizado

- Outras definições de recompensa total são exploradas.
- Exemplos:
  - Recompensa do horizonte finito: considera a soma de recompensas (sem descontos) sobre um número finito  $h$  de passos.

$$\sum_{i=0}^h r_{t+i}$$

- Recompensa média: considera a recompensa média por passo de tempo sobre o tempo de vida toda do agente.

$$\lim_{h \rightarrow \infty} \frac{1}{h} \sum_{i=0}^h r_{t+i}$$

# A Tarefa do Aprendizado

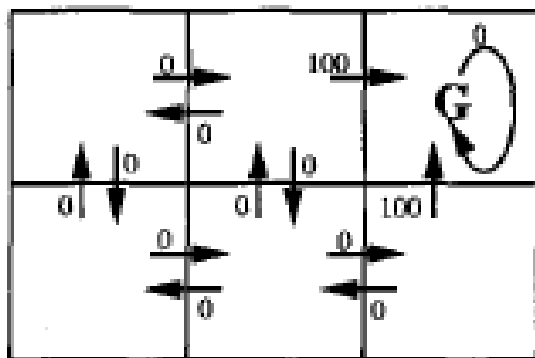
- Nós precisamos que o agente aprenda uma política  $\pi$  que maximize  $V^\pi(s)$  para todos os estados  $s$ .
- Nós chamaremos tal política uma **política ótima** e a denotaremos por  $\pi^*$ .

$$\pi^* \equiv \operatorname{argmax}_{\pi} V^\pi(s), (\forall s)$$

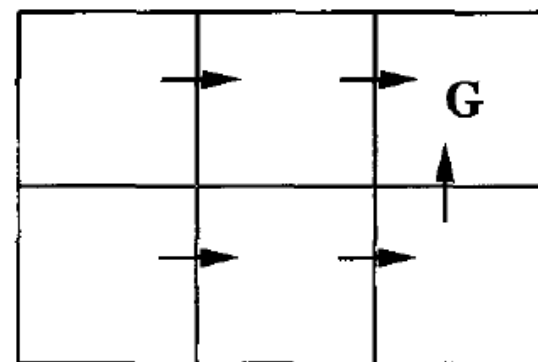
- Para simplificar a notação,  $V^{\pi^*}(s)$  será  $V^*(s)$ :
  - $V^*(s)$  dá a recompensa máxima acumulada com desconto que o agente pode obter começando no estado  $s$  (ou seja, seguindo a política ótima).



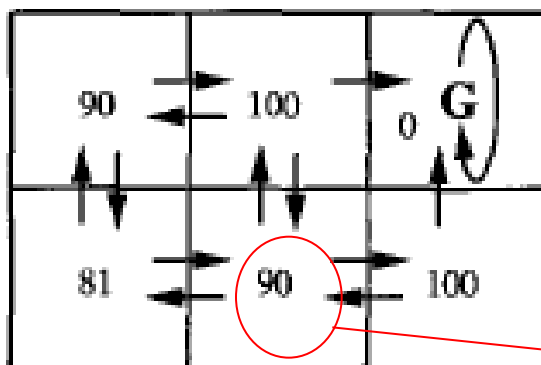
# A Tarefa do Aprendizado



$r(s, a)$  (immediate reward) values



One optimal policy



$V^*(s)$  values

Supondo  $\gamma = 0.9$ .

$$0 + \gamma 100 + \gamma^2 0 + \gamma^3 0 + \dots = 90$$

# Aprendizado Q

- Como um agente pode aprender uma política ótima  $\pi^*$  para um ambiente arbitrário?
  - É difícil aprender a função  $\pi^*$ :  $S \rightarrow A$  diretamente, porque os dados de treinamento disponíveis não fornecem exemplos de treinamento da forma  $\langle s, a \rangle$
  - Em vez disso, as únicas informações de treinamento é a sequência de recompensas imediatas  $r(s_i, a_i)$  para  $i = 0, 1, 2, \dots$
  - Dado esse tipo de informação de treinamento, é mais fácil aprender uma função de avaliação numérica definida sobre os estados e ações. Então, implementamos a política ótima em termos dessa função de avaliação.

# Aprendizado Q

- Uma função de avaliação que o agente deveria tentar aprender é  $V^*$ .
- O agente deveria preferir o estado  $s_1$  ao estado  $s_2$  sempre que  $V^*(s_1) > V^*(s_2)$ .
  - A recompensa futura será maior a partir de  $s_1$ .
- É claro que a política do agente deve escolher entre ações e não entre estados.
  - Contudo, ele pode usar  $V^*$  em certas configurações para escolher entre ações também.

# Aprendizado Q

- A ação ótima no estado  $s$  é a ação  $a$  que maximiza a soma das recompensas imediatas  $r(s, a)$  mais o valor  $V^*$  do estado sucessor imediato, descontado por  $\gamma$ :

$$\pi^*(s) = \underset{a}{\operatorname{argmax}} [r(s, a) + \gamma V^*(\delta(s, a))]$$



**Estado resultante da aplicação da ação  $a$  ao estado  $s$ .**

# Aprendizado Q

- Infelizmente, aprender  $V^*$  é uma maneira útil de aprender a política ótima apenas quando o agente tem conhecimento perfeito de  $\delta$  e  $r$ .
- Em muitos problemas práticos, tal como controle de robô, é impossível prever a saída exata da aplicação de uma ação arbitrária a um estado arbitrário.
- Nos casos onde  $\delta$  ou  $r$  é desconhecido, aprender  $V^*$  não serve para selecionar ações ótimas porque o agente não pode avaliar a equação anterior (eq. do slide anterior – slide 20).
- Nesta configuração mais geral, o agente deveria usar a função de avaliação Q.

# A Função Q

- **Função  $Q(s, a)$ :** é a recompensa acumulada máxima com desconto que pode ser alcançada a partir do estado  $s$  ao aplicar a ação  $a$  como a primeira ação.
- O valor de  $Q$  é a recompensa recebida imediatamente ao executar a ação  $a$  a partir do estado  $s$ , mais o valor (descontado por  $\gamma$ ) da seguinte política ótima:

$$Q(s, a) \equiv r(s, a) + \gamma V^*(\delta(s, a))$$

# A Função Q

- Note que  $Q(s, a)$  é exatamente a quantidade que é maximizada na equação:

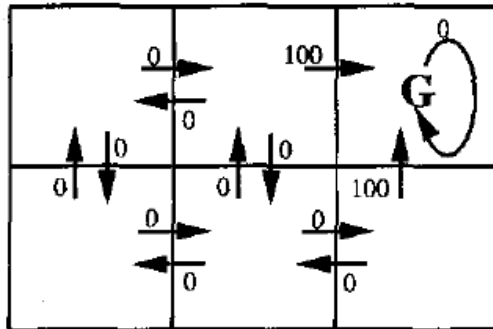
$$\pi^*(s) = \operatorname{argmax}_a [r(s, a) + \gamma V^*(\delta(s, a))]$$

- Portanto, podemos reescrevê-la como:

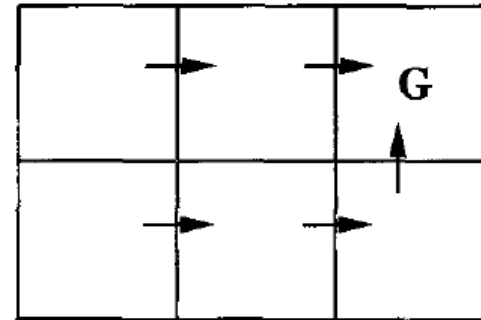
$$\pi^*(s) = \operatorname{argmax}_a Q(s, a)$$

- Por que essa reescrita é importante?
  - Porque mostra que se o agente aprende a função Q em vez da função  $V^*$ , ele será capaz de selecionar ações ótimas mesmo quando não tem conhecimento das funções  $r$  e  $\delta$ .

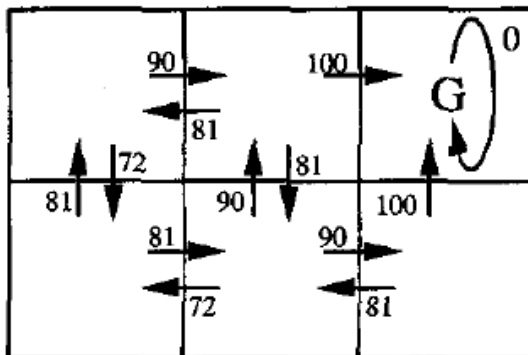
# A Função Q



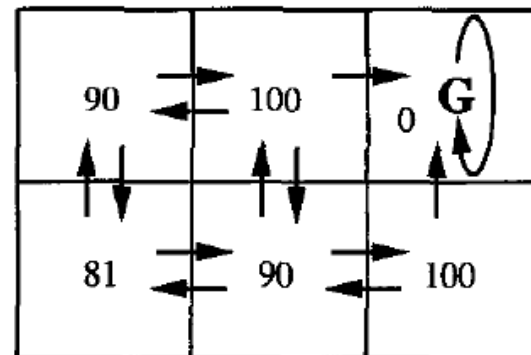
$r(s, a)$  (immediate reward) values



One optimal policy



$Q(s, a)$  values



$V^*(s)$  values



# Um Algoritmo para Aprender Q

- Aprender a função  $Q$  corresponde a aprender a política ótima.
- Como  $Q$  pode ser aprendida?
  - O problema principal é encontrar uma maneira confiável de estimar os valores de treinamento para  $Q$ , dado apenas uma sequência de recompensas imediatas  $r$  espalhadas pelo tempo.
  - Isso pode ser feito através de uma aproximação iterativa.

# Um Algoritmo para Aprender Q

- Note a relação entre  $Q$  e  $V^*$

$$V^*(s) = \max_{a'} Q(s, a')$$

que permite reescrever a função  $Q$  como:

$$Q(s, a) = r(s, a) + \gamma \max_{a'} Q(\delta(s, a), a')$$

- Essa definição recursiva de  $Q$  proporciona a base para algoritmos que iterativamente aproximam  $Q$ .
- No algoritmo, é usado o símbolo  $\hat{Q}$  para referir-se a estimativa do indutor, ou hipótese, da atual função  $Q$ .

# Um Algoritmo para Aprender $Q$

- No algoritmo, o indutor representa sua hipótese  $\hat{Q}$  por uma tabela grande com uma entrada separada para cada par de estado-ação.
- A entrada da tabela para o par  $\langle s, a \rangle$  armazena o valor para  $\hat{Q}(s, a)$  – a hipótese atual do indutor sobre o real porém desconhecido valor  $Q(s, a)$ .

# Um Algoritmo para Aprender Q

- A tabela pode ser inicialmente preenchida com valores aleatórios.
- O agente observa seu estado atual  $s$ , escolhe alguma ação  $a$ , executa essa ação, então observa a recompensa resultante e o novo estado.
- Ele atualiza a entrada da tabela para  $\hat{Q}(s, a)$  para cada transição, de acordo com a regra:

$$\hat{Q}(s, a) \leftarrow r + \gamma \max_{a'} \hat{Q}(s', a')$$

# Um Algoritmo para Aprender Q

---

## Q learning algorithm

For each  $s, a$  initialize the table entry  $\hat{Q}(s, a)$  to zero.

Observe the current state  $s$

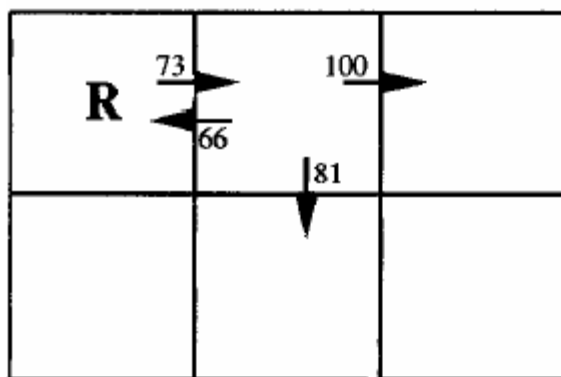
Do forever:

- Select an action  $a$  and execute it
- Receive immediate reward  $r$
- Observe the new state  $s'$
- Update the table entry for  $\hat{Q}(s, a)$  as follows:

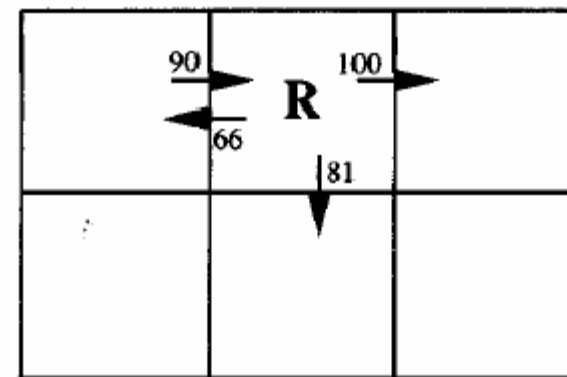
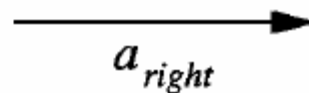
$$\hat{Q}(s, a) \leftarrow r + \gamma \max_{a'} \hat{Q}(s', a')$$

- $s \leftarrow s'$
-

# Um Exemplo Ilustrativo



Initial state:  $s_1$



Next state:  $s_2$

$$\begin{aligned}\hat{Q}(s_1, a_{right}) &\leftarrow r + \gamma \max_{a'} \hat{Q}(s_2, a') \\ &\leftarrow 0 + 0.9 \max\{66, 81, 100\} \\ &\leftarrow 90\end{aligned}$$

# Relação com Programação Dinâmica

- Métodos de aprendizado por reforço estão relacionados a uma linha de pesquisa sobre abordagens de programação dinâmica para resolver processos de decisão de Markov (MDPs).
- A correspondência entre essas abordagens e os problemas de aprendizado por reforço é evidente ao considerar a equação de Bellman, que constitui a base para muitas abordagens de programação dinâmica para resolver MDPs.
- Equação de Bellman:

$$(\forall s \in S) V^*(s) = E[r(s, \pi(s)) + \gamma V^*(\delta(s, \pi(s)))]$$

# Para Estudar!

## Fonte de Referência:

- Livro texto: Machine Learning, de Tom Mitchell: capítulo 13.
- Inteligência artificial, de Russel e Norvig: capítulo 21.